Week 6 Submission

Peer-to-Peer Lending Matchmaking System

Regd No	25091A32G9, 25091A32K4
Problem Statement No	105
Title	Peer-to-Peer Lending Matchmaking System
Technology	Python, Flask, SQLite, NetworkX, Pandas/NumPy, ReportLab, OpenPyXL, Chart.js, Bootstrap, unittest/pytest
Data	Academic simulation using simulated test data - no real-money transactions

Where to run the code (read this first)

- **Where:** on your own laptop. Install **Python 3.10 or newer** from python.org (tick *Add Python to PATH*) and **VS Code**. Everything below is typed in the **VS Code terminal** (menu Terminal > New Terminal, or Ctrl+`).
- **Which folder:** open your **p2p_lending** folder (the one that contains **app.py**) with File > Open Folder. Every command must be run from this folder, not from a sub-folder, not in IDLE.
- **One-time setup:** run `python -m venv venv`, then activate it - Windows: `venv\Scripts\activate` (if PowerShell blocks it use Command Prompt or run `Set-ExecutionPolicy -Scope Process Bypass`), macOS/Linux: `source venv/bin/activate`. Then `pip install -r requirements.txt`.
- **Adding the files of a week:** in VS Code right-click the folder > New File, type the exact path shown in the heading (for example `routes/profiles.py`), paste the code. **[NEW]** = create the file, **[REPLACE]** = overwrite the old file completely, **[APPEND]** = paste at the very end of the existing file.
- **Web pages:** run `python app.py`, then open `http://127.0.0.1:5000` in your browser (stop the server with Ctrl+C). The pages load Bootstrap/Chart.js from the internet, so stay online.
- **Keep the weeks cumulative:** Week N needs the files of Weeks 1 to N-1. Your Week 1 zip is the starting point and none of its files is deleted later (only app.py and templates/index.html are replaced in Week 2, and config.py gets lines appended in Week 3).
- **Demo scripts** (`demo_weekN.py`) create their own throw-away databases in the data folder, so they never disturb your real database. To see the same demo data in the web pages: stop the server, delete `data/p2p_lending.db`, run `python demo_data.py`, start the server again.

Week 6: Dashboard and Integration

Connect the frontend, backend, matching and database modules into one web application with lender, borrower and administrator dashboards, a REST API and downloadable reports. **Working output: integrated web application.**

Assignment steps and where they are done

Assignment step	Implemented in
Lender dashboard	/lender/<id>: funds, total lent, expected interest, portfolio chart, matches and loans
Borrower dashboard	/borrower/<id>: profile, latest risk score and explanation, requests, loans, instalment chart
Administrator dashboard	/admin: system statistics, 4 Chart.js charts, links to every lender and borrower dashboard, report downloads
Match recommendation screen	/matching/recommendations (Week 4 page, part of the menu)
Risk explanation screen	/scoring/risk and the borrower dashboard
Loan and repayment tracker	/loans and /loans/<id>
Reports and audit screen	/reports/audit (filter ?entity_type=loan) + CSV, Excel and PDF downloads
Integration chain: web interface - Flask REST API - modules - database	routes/api.py: JSON endpoints under /api/ that call profile, scoring, matching and loan services

Files of this week

File	Action	Purpose
reports.py	NEW	Dashboard statistics, CSV, Excel (OpenPyXL) and PDF (ReportLab) builders
templates/dashboard.html	NEW	Stat cards, Chart.js charts, tables
routes/dashboard.py, routes/reports.py, routes/api.py	NEW	Dashboards, audit + downloads, REST API
demo_week6.py	NEW	Drives the whole system through the REST API
tests/test_week6.py	NEW	4 tests (dashboards, downloads, complete API flow)

REST API (all under /api)

Method and path	Purpose
GET /lenders, /borrowers, /requests, /risk-scores, /matches?status=, /loans, /loans/<id>/repayments, /stats	Read data as JSON
POST /lenders, /borrowers, /requests	Register (JSON body, same validation as the web forms; errors return HTTP 400)
POST /profiles/<role>/<id>/status	Body {"status": "active"}
POST /scoring/run, /matching/run	Body {"partial": true} optional
POST /loans/accept/<request_id>, /loans/<id>/simulate	Create loans; simulate repayments (body {"scenario": "all_paid"})

How to run and check this week

1. `python tests/test_week6.py` - 4 tests, all ok
2. `python demo_week6.py` - calls the REST API and opens every screen and download
3. `python app.py` - open /admin and click the Dashboard buttons; download the CSV / Excel / PDF reports

Expected output of demo_week6.py

```
--- REST API walk-through ---
GET  /api/stats                          -> HTTP 200
{'Active borrowers': 6, 'Active lenders': 5, 'Active loans': 0, 'Audit entries': 18, 'Defaulted loans': 0, 'Funded requests': 0, 'Matched
    requests': 0, 'Pending requests': 7, 'Recommended matches': 0, 'Total outstanding': '0', 'Total principal': '0'}
POST /api/matching/run                   -> HTTP 200
{'allocation_success_rate': 71.4, 'eligible_edges': 11, 'lenders': 5, 'partial_funding': False, 'recommended': 5, 'requests': 7,
    'requests_matched': 5, 'seconds': 0.0073, 'total_allocated': 285000.0}
POST /api/loans/accept/1                 -> HTTP 201
{'loan_ids': [1]}
POST /api/loans/accept/2                 -> HTTP 201
{'loan_ids': [2]}
POST /api/loans/accept/3                 -> HTTP 201
{'loan_ids': [3]}
POST /api/loans/1/simulate (default)     -> HTTP 200
defaulted
POST /api/lenders (invalid)              -> HTTP 400
['Available amount is required.', 'Minimum loan amount is required.']

--- Screens ---
GET  /                                   -> HTTP 200
GET  /admin                              -> HTTP 200
GET  /lender/1                           -> HTTP 200
GET  /borrower/1                         -> HTTP 200
GET  /reports/audit                      -> HTTP 200
GET  /loans                              -> HTTP 200
GET  /loans/1                            -> HTTP 200
GET  /scoring/pairs                      -> HTTP 200
GET  /scoring/risk                       -> HTTP 200
GET  /matching/recommendations           -> HTTP 200
GET  /loans/pending                      -> HTTP 200
GET  /lenders                            -> HTTP 200
GET  /borrowers                          -> HTTP 200
GET  /requests                           -> HTTP 200
GET  /reports/loans.csv                  -> HTTP 200
     downloaded 384 bytes (text/csv)
GET  /reports/loans.xlsx                 -> HTTP 200
     downloaded 10,194 bytes (application/vnd.openxmlformats-officedocument.spreadsheetml.sheet)
GET  /reports/summary.pdf                -> HTTP 200
     downloaded 2,847 bytes (application/pdf)
```

[NEW] reports.py

```
"""Week 6 - dashboard statistics and CSV / Excel / PDF reports."""
import io

import pandas as pd
from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer, Table, TableStyle

import loan_service
import profile_service
import queries


def _one(conn, sql, params=()):
    value = conn.execute(sql, params).fetchone()[0]
    return value if value is not None else 0


def _counts(conn, sql, params=()):
    rows = conn.execute(sql, params).fetchall()
    return [r[0] for r in rows], [round(r[1], 2) for r in rows]


# ---------------------------------------------------------------------------
# administrator
# ---------------------------------------------------------------------------
def system_stats(conn):
    return [
        ("Active lenders", _one(conn, "SELECT COUNT(*) FROM lenders WHERE profile_status='active'")),
        ("Active borrowers", _one(conn, "SELECT COUNT(*) FROM borrowers WHERE profile_status='active'")),
        ("Pending requests", _one(conn, "SELECT COUNT(*) FROM loan_requests WHERE status='pending'")),
        ("Matched requests", _one(conn, "SELECT COUNT(*) FROM loan_requests WHERE status='matched'")),
        ("Funded requests", _one(conn, "SELECT COUNT(*) FROM loan_requests WHERE status='funded'")),
        ("Recommended matches", _one(conn, "SELECT COUNT(*) FROM matches WHERE status='recommended' AND allocated_amount>0")),
        ("Active loans", _one(conn, "SELECT COUNT(*) FROM loans WHERE status='active'")),
        ("Defaulted loans", _one(conn, "SELECT COUNT(*) FROM loans WHERE status='defaulted'")),
        ("Total principal", f"{_one(conn, 'SELECT SUM(principal) FROM loans'):,.0f}"),
        ("Total outstanding", f"{_one(conn, 'SELECT SUM(outstanding_balance) FROM loans'):,.0f}"),
        ("Audit entries", _one(conn, "SELECT COUNT(*) FROM audit_logs")),
    ]


def admin_charts(conn):
    risk_labels, risk_data = _counts(conn, "SELECT risk_category, COUNT(*) FROM risk_scores GROUP BY risk_category")
    match_labels, match_data = _counts(conn, "SELECT status, COUNT(*) FROM matches GROUP BY status")
    rep_labels, rep_data = _counts(conn, "SELECT status, COUNT(*) FROM repayments GROUP BY status")
    loan_labels, loan_data = _counts(conn, "SELECT status, COUNT(*) FROM loans GROUP BY status")
    return [
        {"id": "riskChart", "title": "Borrower risk classes", "type": "doughnut", "labels": risk_labels, "data": risk_data},
        {"id": "matchChart", "title": "Scored pairs by status", "type": "bar", "labels": match_labels, "data": match_data},
        {"id": "repayChart", "title": "Repayment statuses", "type": "bar", "labels": rep_labels, "data": rep_data},
        {"id": "loanChart", "title": "Loans by status", "type": "doughnut", "labels": loan_labels, "data": loan_data},
    ]
# ---------------------------------------------------------------------------
```

```
# lender
# ---------------------------------------------------------------------------
def lender_dashboard(conn, lender_id):
    lender = profile_service.get_lender(conn, lender_id)
    if lender is None:
        return None
    lent = _one(conn, "SELECT SUM(principal) FROM loans WHERE lender_id = ?", (lender_id,))
    interest = _one(conn, "SELECT SUM(r.amount_due) FROM repayments r JOIN loans l ON l.loan_id = r.loan_id "
                          "WHERE l.lender_id = ?", (lender_id,)) - lent if lent else 0
    defaulted = _one(conn, "SELECT SUM(principal) FROM loans WHERE lender_id = ? AND status = 'defaulted'", (lender_id,))
    stats = [("Risk appetite", lender.risk_appetite.capitalize()), ("Profile status", lender.profile_status),
             ("Available funds", f"{lender.available_amount:,.0f}"), ("Total lent", f"{lent:,.0f}"),
             ("Active loans", _one(conn, "SELECT COUNT(*) FROM loans WHERE lender_id=? AND status='active'", (lender_id,))),
             ("Expected interest (scheduled)", f"{interest:,.0f}"),
             ("Defaulted principal", f"{defaulted:,.0f}")]
    r_labels, r_data = _counts(
        conn, "SELECT COALESCE(s.risk_category,'unknown'), SUM(lo.principal) FROM loans lo "
              "JOIN matches m ON m.match_id = lo.match_id LEFT JOIN risk_scores s ON s.request_id = m.request_id "
              "WHERE lo.lender_id = ? GROUP BY 1", (lender_id,))
    p_labels, p_data = _counts(
        conn, "SELECT r.status, COUNT(*) FROM repayments r JOIN loans l ON l.loan_id = r.loan_id "
              "WHERE l.lender_id = ? GROUP BY r.status", (lender_id,))
    charts = [{"id": "portfolio", "title": "Amount lent by borrower risk class", "type": "doughnut",
               "labels": r_labels, "data": r_data},
              {"id": "repay", "title": "Instalment statuses of my loans", "type": "bar",
               "labels": p_labels, "data": p_data}]
    matches = [m for m in queries.list_pairs(conn, lender_id=lender_id)
               if m["status"] in ("recommended", "accepted") and m["allocated_amount"] > 0]
    loans = [l for l in loan_service.list_loans(conn) if l["lender_id"] == lender_id]
    return {"title": f"Lender dashboard - {lender.full_name}", "stats": stats, "charts": charts,
            "sections": [
                {"title": "Recommended / accepted matches", "rows": matches,
                 "columns": [("request_id", "Req"), ("borrower", "Borrower"), ("allocated_amount", "Allocated"),
                             ("score", "Score"), ("status", "Status"), ("explanation", "Why")]},
                {"title": "My loans", "rows": loans,
                 "columns": [("loan_id", "Loan"), ("borrower", "Borrower"), ("principal", "Principal"),
                             ("interest_rate", "Rate %"), ("duration_months", "Months"),
                             ("outstanding_balance", "Outstanding"), ("status", "Status")]}]}


# ---------------------------------------------------------------------------
# borrower
# ---------------------------------------------------------------------------
def borrower_dashboard(conn, borrower_id):
    borrower = profile_service.get_borrower(conn, borrower_id)
    if borrower is None:
        return None
    risk = conn.execute("SELECT score, risk_category FROM risk_scores WHERE borrower_id = ? "
                        "ORDER BY risk_score_id DESC LIMIT 1", (borrower_id,)).fetchone()
    stats = [("Profile status", borrower.profile_status), ("Monthly income", f"{borrower.monthly_income:,.0f}"),
             ("Obligations", f"{borrower.existing_obligations:,.0f}"), ("Debt-to-income", f"{borrower.debt_to_income():.0%}"),
             ("Credit score", borrower.credit_score),
             ("Latest risk score", f"{risk['score']} ({risk['risk_category']})" if risk else "not scored yet"),
             ("Total borrowed", f"{_one(conn, 'SELECT SUM(principal) FROM loans WHERE borrower_id=?', (borrower_id,)):,.0f}"),
             ("Outstanding", f"{_one(conn, 'SELECT SUM(outstanding_balance) FROM loans WHERE borrower_id=?', (borrower_id,)):,.0f}")]
    p_labels, p_data = _counts(
        conn, "SELECT r.status, COUNT(*) FROM repayments r JOIN loans l ON l.loan_id = r.loan_id "
              "WHERE l.borrower_id = ? GROUP BY r.status", (borrower_id,))
    requests = [r for r in profile_service.list_requests(conn) if r["borrower_id"] == borrower_id]
    loans = [l for l in loan_service.list_loans(conn) if l["borrower_id"] == borrower_id]
    risks = [dict(r) for r in conn.execute(
        "SELECT request_id, score, risk_category, explanation FROM risk_scores WHERE borrower_id = ?", (borrower_id,))]
    return {"title": f"Borrower dashboard - {borrower.full_name}", "stats": stats,
            "charts": [{"id": "repay", "title": "My instalment statuses", "type": "bar",
                        "labels": p_labels, "data": p_data}],
            "sections": [
                {"title": "My loan requests", "rows": requests,
                 "columns": [("request_id", "ID"), ("amount", "Amount"), ("term_months", "Term"),
                             ("purpose", "Purpose"), ("status", "Status")]},
                {"title": "Why my risk score is what it is", "rows": risks,
                 "columns": [("request_id", "Req"), ("score", "Score"), ("risk_category", "Class"),
                             ("explanation", "Explanation")]},
                {"title": "My loans", "rows": loans,
                 "columns": [("loan_id", "Loan"), ("lender", "Lender"), ("principal", "Principal"),
                             ("interest_rate", "Rate %"), ("outstanding_balance", "Outstanding"), ("status", "Status")]}]}


# ---------------------------------------------------------------------------
# audit + downloads
# ---------------------------------------------------------------------------
def audit_rows(conn, entity_type=None, limit=300):
    sql, params = "SELECT * FROM audit_logs", []
    if entity_type:
        sql += " WHERE entity_type = ?"
        params.append(entity_type)
    sql += " ORDER BY log_id DESC LIMIT ?"
    params.append(limit)
    return [dict(r) for r in conn.execute(sql, params)]


def loans_dataframe(conn):
    return pd.DataFrame(loan_service.list_loans(conn))


def build_csv(conn):
    return loans_dataframe(conn).to_csv(index=False)


def build_xlsx(conn):
    buffer = io.BytesIO()
```

```
    with pd.ExcelWriter(buffer, engine="openpyxl") as writer:
        loans_dataframe(conn).to_excel(writer, sheet_name="Loans", index=False)
        pd.DataFrame([dict(r) for r in conn.execute("SELECT * FROM repayments")]).to_excel(
            writer, sheet_name="Repayments", index=False)
        pd.DataFrame(audit_rows(conn, limit=5000)).to_excel(writer, sheet_name="Audit", index=False)
    return buffer.getvalue()


def build_pdf(conn):
    buffer = io.BytesIO()
    doc = SimpleDocTemplate(buffer, pagesize=A4, title="P2P Lending Simulation Report")
    styles = getSampleStyleSheet()
    story = [Paragraph("P2P Lending Simulation Report", styles["Title"]),
             Paragraph("Academic simulation - no real money involved.", styles["Italic"]), Spacer(1, 12)]

    def table(data, widths=None):
        t = Table(data, colWidths=widths, repeatRows=1)
        t.setStyle(TableStyle([("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#0d6efd")),
                               ("TEXTCOLOR", (0, 0), (-1, 0), colors.white), ("FONTSIZE", (0, 0), (-1, -1), 8),
                               ("GRID", (0, 0), (-1, -1), 0.25, colors.grey)]))
        return t
    story += [Paragraph("System summary", styles["Heading2"]),
              table([["Measure", "Value"]] + [[a, str(b)] for a, b in system_stats(conn)], [250, 150]), Spacer(1, 14),
              Paragraph("Loans", styles["Heading2"])]
    loans = loan_service.list_loans(conn)
    rows = [["Loan", "Lender", "Borrower", "Principal", "Rate %", "Months", "Outstanding", "Status"]]
    rows += [[l["loan_id"], l["lender"], l["borrower"], f"{l['principal']:,.0f}", l["interest_rate"],
              l["duration_months"], f"{l['outstanding_balance']:,.0f}", l["status"]] for l in loans[:60]]
    story.append(table(rows) if loans else Paragraph("No loans yet.", styles["Normal"]))
    doc.build(story)
    return buffer.getvalue()
```

[NEW] templates/dashboard.html

```
{% extends "base.html" %}
{% block content %}
<div class="d-flex justify-content-between align-items-center mb-3">
  <h2 class="h4 mb-0">{{ title }}</h2>
  <div class="d-flex gap-2">
    {% for label, url in downloads or [] %}<a class="btn btn-sm btn-outline-primary" href="{{ url }}">{{ label }}</a>{% endfor %}
  </div>
</div>
<div class="row g-3 mb-4">
  {% for label, value in stats %}
  <div class="col-6 col-md-3"><div class="card shadow-sm h-100"><div class="card-body py-3">
    <div class="text-muted small">{{ label }}</div><div class="fs-5 fw-semibold">{{ value }}</div>
  </div></div></div>
  {% endfor %}
</div>
{% if charts %}
<div class="row g-3 mb-4">
  {% for c in charts %}
  <div class="col-md-6"><div class="card shadow-sm"><div class="card-body">
    <h3 class="h6">{{ c.title }}</h3>
    {% if c.data %}<canvas id="{{ c.id }}" height="160"></canvas>{% else %}<p class="text-muted mb-0">No data yet.</p>{% endif %}
  </div></div></div>
  {% endfor %}
</div>
{% endif %}
{% macro button(a) %}
  <a class="btn btn-sm btn-{{ a.style or 'outline-primary' }}" href="{{ a.url }}">{{ a.label }}</a>
{% endmacro %}
{% for s in sections or [] %}
<div class="card shadow-sm mb-4">
  <div class="card-header">{{ s.title }}</div>
  <div class="table-responsive"><table class="table table-sm table-hover align-middle mb-0">
    <thead class="table-light"><tr>{% for key, label in s.columns %}<th>{{ label }}</th>{% endfor %}<th></th></tr></thead>
    <tbody>
      {% for row in s.rows %}
      <tr>{% for key, label in s.columns %}<td>{{ '' if row.get(key) is none else row.get(key) }}</td>{% endfor %}
          <td class="text-end">{% for a in row.get('_actions', []) %}{{ button(a) }} {% endfor %}</td></tr>
      {% else %}<tr><td colspan="{{ s.columns|length + 1 }}" class="text-center text-muted py-3">Nothing to show yet.</td></tr>
      {% endfor %}
    </tbody></table></div>
</div>
{% endfor %}
{% endblock %}
{% block scripts %}
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.3/dist/chart.umd.min.js"></script>
<script>
  const charts = {{ charts|tojson }};
  const palette = ['#0d6efd', '#198754', '#ffc107', '#dc3545', '#6f42c1', '#20c997'];
  charts.forEach(c => {
    const el = document.getElementById(c.id);
    if (!el || !c.data.length) return;
    new Chart(el, {type: c.type,
      data: {labels: c.labels, datasets: [{data: c.data, backgroundColor: palette}]},
      options: {plugins: {legend: {display: c.type === 'doughnut'}}}});
  });
</script>
{% endblock %}
```

[NEW] routes/dashboard.py

```
"""Week 6 routes - administrator, lender and borrower dashboards."""
from flask import Blueprint, abort, render_template, url_for

import profile_service as ps
import reports
from utils import db_conn

bp = Blueprint("dashboard", __name__)


@bp.route("/admin")
def admin():
    with db_conn() as conn:
        lenders = [dict(l.to_dict(), _actions=[{"label": "Dashboard", "url": url_for("dashboard.lender", lender_id=l.lender_id)}])
                   for l in ps.list_lenders(conn)]
        borrowers = [dict(b.to_dict(), _actions=[{"label": "Dashboard", "url": url_for("dashboard.borrower", borrower_id=b.borrower_id)}])
                     for b in ps.list_borrowers(conn)]
        stats, charts = reports.system_stats(conn), reports.admin_charts(conn)
    sections = [
        {"title": "Lenders", "rows": lenders, "columns": [("lender_id", "ID"), ("full_name", "Name"),
         ("risk_appetite", "Risk"), ("available_amount", "Available"), ("profile_status", "Status")]},
        {"title": "Borrowers", "rows": borrowers, "columns": [("borrower_id", "ID"), ("full_name", "Name"),
         ("monthly_income", "Income"), ("credit_score", "Credit"), ("profile_status", "Status")]}]
    downloads = [("Loans CSV", url_for("reports.loans_csv")), ("Excel report", url_for("reports.loans_xlsx")),
                 ("PDF report", url_for("reports.summary_pdf"))]
    return render_template("dashboard.html", title="Administrator dashboard", stats=stats, charts=charts,
                           sections=sections, downloads=downloads)


@bp.route("/lender/<int:lender_id>")
def lender(lender_id):
    with db_conn() as conn:
        data = reports.lender_dashboard(conn, lender_id)
    if data is None:
        abort(404)
    return render_template("dashboard.html", **data)


@bp.route("/borrower/<int:borrower_id>")
def borrower(borrower_id):
    with db_conn() as conn:
        data = reports.borrower_dashboard(conn, borrower_id)
    if data is None:
        abort(404)
    return render_template("dashboard.html", **data)
```

[NEW] routes/reports.py

```
"""Week 6 routes - audit screen and downloadable CSV / Excel / PDF reports."""
from flask import Blueprint, Response, render_template, request

import reports
from utils import db_conn

bp = Blueprint("reports", __name__, url_prefix="/reports")


@bp.route("/audit")
def audit_log():
    entity = request.args.get("entity_type")
    with db_conn() as conn:
        rows = reports.audit_rows(conn, entity_type=entity)
    columns = [("log_id", "#"), ("created_at", "Time"), ("entity_type", "Entity"), ("entity_id", "ID"),
               ("action", "Action"), ("old_value", "Old"), ("new_value", "New / details"), ("actor", "Actor")]
    return render_template("table.html", title="Audit log (latest 300)", columns=columns, rows=rows,
                           intro="Filter with ?entity_type=loan, repayment, match, loan_request, lender, borrower or system.")


@bp.route("/loans.csv")
def loans_csv():
    with db_conn() as conn:
        data = reports.build_csv(conn)
    return Response(data, mimetype="text/csv", headers={"Content-Disposition": "attachment; filename=loans.csv"})


@bp.route("/loans.xlsx")
def loans_xlsx():
    with db_conn() as conn:
        data = reports.build_xlsx(conn)
    return Response(data, headers={"Content-Disposition": "attachment; filename=p2p_report.xlsx"},
                    mimetype="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")
@bp.route("/summary.pdf")
def summary_pdf():
    with db_conn() as conn:
        data = reports.build_pdf(conn)
    return Response(data, mimetype="application/pdf", headers={"Content-Disposition": "inline; filename=p2p_report.pdf"})
```

[NEW] routes/api.py

```
"""Week 6 - Flask REST API: the web interface and external tools talk to the
profile, scoring, matching and loan modules through these JSON endpoints."""
from flask import Blueprint, jsonify, request

import loan_service as ls
import profile_service as ps
import queries
import reports
import scoring
from matching_engine import MatchingEngine
from models import ValidationError
from utils import db_conn

bp = Blueprint("api", __name__, url_prefix="/api")


@bp.errorhandler(ValidationError)
def handle_validation_error(exc):
    return jsonify(error="validation_failed", messages=exc.errors), 400


def _body():
    return request.get_json(silent=True) or {}


# ---- read -----------------------------------------------------------------
@bp.route("/lenders")
def lenders():
    with db_conn() as conn:
        return jsonify([l.to_dict() for l in ps.list_lenders(conn)])


@bp.route("/borrowers")
def borrowers():
    with db_conn() as conn:
        return jsonify([b.to_dict() for b in ps.list_borrowers(conn)])


@bp.route("/requests")
def requests_list():
    with db_conn() as conn:
        return jsonify(ps.list_requests(conn, status=request.args.get("status")))


@bp.route("/risk-scores")
def risk_scores():
    with db_conn() as conn:
        return jsonify(queries.list_risk_scores(conn))


@bp.route("/matches")
def matches():
    with db_conn() as conn:
        return jsonify(queries.list_pairs(conn, status=request.args.get("status"),
                                          request_id=request.args.get("request_id", type=int)))


@bp.route("/loans")
def loans():
    with db_conn() as conn:
        return jsonify(ls.list_loans(conn))


@bp.route("/loans/<int:loan_id>/repayments")
def repayments(loan_id):
    with db_conn() as conn:
        if ls.get_loan(conn, loan_id) is None:
            return jsonify(error="not_found"), 404
        return jsonify(ls.get_repayments(conn, loan_id))


@bp.route("/stats")
def stats():
    with db_conn() as conn:
        return jsonify({label: value for label, value in reports.system_stats(conn)})


# ---- write ----------------------------------------------------------------
@bp.route("/lenders", methods=["POST"])
def create_lender():
    with db_conn() as conn:
        return jsonify(lender_id=ps.register_lender(conn, _body())), 201


@bp.route("/borrowers", methods=["POST"])
def create_borrower():
    with db_conn() as conn:
        return jsonify(borrower_id=ps.register_borrower(conn, _body())), 201


@bp.route("/profiles/<role>/<int:profile_id>/status", methods=["POST"])
def profile_status(role, profile_id):
    with db_conn() as conn:
        ps.set_profile_status(conn, role, profile_id, _body().get("status", ""))
        return jsonify(ok=True)


@bp.route("/requests", methods=["POST"])
```

```
def create_request():
    data = _body()
    with db_conn() as conn:
        try:
            borrower_id = int(data.get("borrower_id"))
        except (TypeError, ValueError):
            raise ValidationError("borrower_id is required.")
        return jsonify(request_id=ps.create_loan_request(conn, borrower_id, data)), 201


@bp.route("/scoring/run", methods=["POST"])
def run_scoring():
    with db_conn() as conn:
        return jsonify(scoring.score_all(conn, allow_partial=bool(_body().get("partial"))))


@bp.route("/matching/run", methods=["POST"])
def run_matching():
    with db_conn() as conn:
        return jsonify(MatchingEngine(conn, allow_partial=bool(_body().get("partial"))).run())


@bp.route("/loans/accept/<int:request_id>", methods=["POST"])
def accept(request_id):
    with db_conn() as conn:
        return jsonify(loan_ids=ls.accept_request(conn, request_id)), 201


@bp.route("/loans/<int:loan_id>/simulate", methods=["POST"])
def simulate(loan_id):
    data = _body()
    with db_conn() as conn:
        return jsonify(ls.simulate_loan(conn, loan_id, data.get("scenario", "random"), data.get("seed")))
```

[NEW] demo_week6.py

```
"""Week 6 demo: drives the whole system through the Flask REST API (uses data/demo6.db)."""
import os

import config
import demo_data
from app import create_app

path = os.path.join(config.BASE_DIR, "data", "demo6.db")
if os.path.exists(path):
    os.remove(path)
client = create_app(path).test_client()      # create_app builds the empty database


def show(title, response):
    print(f"{title:<42} -> HTTP {response.status_code}")
    return response


import database
conn = database.get_connection(path)
demo_data.load_demo_data(conn)
conn.close()

print("--- REST API walk-through ---")
print(show("GET  /api/stats", client.get("/api/stats")).get_json())
print(show("POST /api/matching/run", client.post("/api/matching/run", json={})).get_json())
for rid in (1, 2, 3):
    print(show(f"POST /api/loans/accept/{rid}", client.post(f"/api/loans/accept/{rid}")).get_json())
print(show("POST /api/loans/1/simulate (default)", client.post("/api/loans/1/simulate", json={"scenario": "default"})).get_json()["status"])
bad = show("POST /api/lenders (invalid)", client.post("/api/lenders", json={"full_name": "x"}))
print(bad.get_json()["messages"][:2])

print("\n--- Screens ---")
for url in ("/", "/admin", "/lender/1", "/borrower/1", "/reports/audit", "/loans", "/loans/1", "/scoring/pairs",
            "/scoring/risk", "/matching/recommendations", "/loans/pending", "/lenders", "/borrowers", "/requests"):
    show("GET  " + url, client.get(url))
for url in ("/reports/loans.csv", "/reports/loans.xlsx", "/reports/summary.pdf"):
    r = show("GET  " + url, client.get(url))
    print(f"      downloaded {len(r.data):,} bytes ({r.mimetype})")
```

[NEW] tests/test_week6.py

```
"""Week 6 tests: dashboards, REST API integration, downloadable reports."""
import json
import os
import sys
import unittest

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from helpers import add_borrower, add_lender, add_request, new_conn
import loan_service as ls
from app import create_app
from matching_engine import MatchingEngine


def world():
    conn, path = new_conn()
    lender = add_lender(conn, available=300000)
    borrower = add_borrower(conn)
    rid = add_request(conn, borrower, amount=60000, term=12)
    MatchingEngine(conn).run()
    loan_id = ls.accept_request(conn, rid)[0]
    ls.simulate_loan(conn, loan_id, "some_late")
    conn.close()
    return create_app(path).test_client(), lender, borrower, loan_id


class DashboardTests(unittest.TestCase):
    def test_all_dashboards_and_screens_open(self):
        client, lender, borrower, loan_id = world()
        for url in ("/admin", f"/lender/{lender}", f"/borrower/{borrower}", "/reports/audit",
                    "/reports/audit?entity_type=loan", "/loans", f"/loans/{loan_id}"):
            self.assertEqual(client.get(url).status_code, 200, url)
        self.assertEqual(client.get("/lender/999").status_code, 404)
        self.assertEqual(client.get("/borrower/999").status_code, 404)
        self.assertIn(b"Administrator dashboard", client.get("/admin").data)
    def test_downloads(self):
        client, *_ = world()
        self.assertIn(b"loan_id", client.get("/reports/loans.csv").data)
        self.assertEqual(client.get("/reports/loans.xlsx").data[:2], b"PK")          # xlsx is a zip file
        self.assertEqual(client.get("/reports/summary.pdf").data[:4], b"%PDF")


class ApiTests(unittest.TestCase):
    def test_full_flow_through_rest_api(self):
        conn, path = new_conn()
        conn.close()
        c = create_app(path).test_client()
        lender = {"full_name": "Api Lender", "email": "al@x.com", "password": "password123", "risk_appetite": "medium",
                  "available_amount": 200000, "min_loan_amount": 5000, "max_loan_amount": 100000,
                  "min_duration_months": 6, "max_duration_months": 36, "min_expected_return": 8, "max_expected_return": 22}
        borrower = {"full_name": "Api Borrower", "email": "ab@x.com", "password": "password123", "monthly_income": 60000,
                    "existing_obligations": 5000, "credit_score": 720, "credit_history_years": 5}
        self.assertEqual(c.post("/api/lenders", json=lender).status_code, 201)
        self.assertEqual(c.post("/api/borrowers", json=borrower).status_code, 201)
        self.assertEqual(c.post("/api/profiles/lender/1/status", json={"status": "active"}).status_code, 200)
        self.assertEqual(c.post("/api/profiles/borrower/1/status", json={"status": "active"}).status_code, 200)
        r = c.post("/api/requests", json={"borrower_id": 1, "amount": 50000, "term_months": 12})
        self.assertEqual(r.status_code, 201)
        self.assertEqual(c.post("/api/requests", json={"borrower_id": 1, "amount": 50000, "term_months": 12}).status_code, 400)
        self.assertEqual(c.post("/api/scoring/run").get_json()["eligible"], 1)
        self.assertEqual(c.post("/api/matching/run", json={}).get_json()["requests_matched"], 1)
        self.assertEqual(len(c.get("/api/matches?status=recommended").get_json()), 1)
        self.assertEqual(c.post("/api/loans/accept/1").get_json()["loan_ids"], [1])
        self.assertEqual(c.post("/api/loans/1/simulate", json={"scenario": "all_paid"}).get_json()["status"], "completed")
        self.assertEqual(len(c.get("/api/loans/1/repayments").get_json()), 12)
        self.assertEqual(c.get("/api/loans/99/repayments").status_code, 404)
        self.assertEqual(c.get("/api/stats").get_json()["Funded requests"], 1)

    def test_validation_errors_are_json(self):
        conn, path = new_conn()
        conn.close()
        r = create_app(path).test_client().post("/api/lenders", json={"full_name": "x"})
        self.assertEqual(r.status_code, 400)
        self.assertEqual(r.get_json()["error"], "validation_failed")


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Screenshots to attach to your submission

- Administrator dashboard (/admin) with the charts.
- Lender dashboard and borrower dashboard.
- Audit screen (/reports/audit) and the downloaded PDF/Excel report.
- Browser showing JSON from http://127.0.0.1:5000/api/stats.